

TP2 Team Norms

Updated from Team1 Norms (TP1) with track changes for TP2

Sutton Harr, Rufat Babayev, Ahad Mammadli

Review note: Changed norm text below uses tracked changes. Each section also includes a short note so the reason for the update is evident and easy to grade or edit later.
--

Coding Style and Package Consistency:

Keep new TP2 code aligned with the provided Foundations structure, but favor shared team conventions over copying every starter-code quirk. Package names, class names, method naming, indentation, spacing, and UI/controller/model/database boundaries should be consistent before parallel work begins.

Keep methods focused and readable, separate responsibilities cleanly, and add short internal comments where the purpose is not obvious from the code alone. When a block of code directly satisfies a requirement or test expectation, leave a brief trace note nearby so later screenshots and Javadoc are easier to produce.

***Change Notes:** TP1 worked best when code stayed close to the provided Foundations structure, but TP2 adds more packages, more parallel work, and much heavier documentation. We tightened this norm so consistency covers architecture and requirement-tracing, not just indentation.*

Meetings, Communication, Sharing, and Status Tracking:

Meet at least twice per week during active project windows, decide the next standup before ending the current one, and post a status update the same day if you cannot attend. Blockers that could affect another teammate or a deliverable should be reported as soon as they are known.

All project work should live in GitHub and on the team tracking board with an owner, current status, target done date, and links to the relevant commit or pull request when possible. Reassignments are allowed / inevitable but the board or plan must be updated the same day so ownership never becomes ambiguous.

***Change Notes:** TP1's plan already showed task reallocation, and TP2 now requires repeated standups plus dated plan/status evidence. The updated norm makes communication and ownership more specific so work does not drift or become ambiguous late in the sprint.*

Documentation, Commenting, and Requirement Traceability:

Each source file should still start with a short purpose description, but TP2 work must also be documented while it is being built rather than after the fact. Javadoc, internal comments, screenshots of key code, and notes about which requirement a method satisfies should be captured as features land, especially for classes and tests that will become PDF deliverables.

***Change Notes:** In TP1, screenshots and writeups were easiest to finish when notes were captured as features landed. TP2 asks for large Javadoc PDFs plus screenshots of internal comments and tests, so documentation should no longer be treated as an end-of-project cleanup step.*

UI/UX and Discussion-Workflow Rules:

- Maintain overall similarity to the provided application so new discussion-board screens feel like part of the same system, not a separate app.
- Keep font choices, spacing, labels, and control placement consistent across all screens, including create, edit, and view flows for posts and replies.
- Validation and status messages should be specific, concise, and shown in a consistent style; destructive actions should use confirmations, and blocked actions should explain why, such as a closed thread, invalid selection, or missing required field.
- Lists, filters, search controls, and action buttons should stay in ‘predictable’ locations and use clean empty-state messages rather than blank or confusing screens.
- Success feedback for important actions such as create, update, delete, resolve or close, or password reset should be visible long enough for the user to notice/acknowledge.
- Standard accessibility should remain part of the default design: readable font sizes, clear contrast, logical tab order where possible, and wording that is easy to understand under stress.

Our TP1 UX and validation documents emphasized consistent feedback, field-level errors, and predictable layouts. TP2 adds more stateful interactions; create/edit/view flows, filters, search, and closed/open thread behavior. Our norms needed to cover those explicitly.

Testing and Deliverable Readiness:

A task is not considered **Done** until the feature runs, its planned manual or automated tests pass, the affected UI and validation states have been checked, and the team has enough saved evidence to document it later. Before submission, each PDF, screencast, or archive should have a primary owner and a second reviewer, and every team member should verify final naming and upload readiness because the TP2 instructions make submission responsibility shared across the whole team.

*TP1 closeout still needed regression passes and final PDF work, which showed that “code compiles” is not the same thing as “submission ready.” Other teams that seemed more organized were treating tests, screenshots, and screencast prep as part of completion, so we added an **explicit** definition of done.*